

Database Normalization

Introduction to Database Management Systems

Anomalies in DBMS

- Anomaly means inconsistency in the pattern from the normal form.
- In Database Management System (DBMS), anomaly means the inconsistency occurred in the relational table during the operations performed on the relational table.
- There can be various reasons for anomalies to occur in the database.
- For example, if there is a lot of redundant data present in our database then DBMS anomalies can occur.
- If a table is constructed in a very poor manner then there is a chance of database anomaly. Due to database anomalies, the integrity of the database suffers.

Anomalies in DBMS

- The other reason for the database anomalies is that all the data is stored in a single table.
- So, to remove the anomalies of the database, normalization is the process which is done where the splitting of the table and joining of the table (different types of join) occurs.

Types of Anomalies in DBMS

- There are three types of anomalies in DBMS
 - Insertion
 - Updating
 - Deletion

Anomalies in DBMS

- **Update Anomaly:** When we update some rows in the table, and if it leads to the inconsistency of the table then this anomaly occurs. This type of anomaly is known as an updation anomaly.
- **Insertion Anomaly:** If there is a new row inserted in the table and it creates the inconsistency in the table then it is called the insertion anomaly.
- **Deletion Anomaly:** If we delete some rows from the table and if any other information or data which is required is also deleted from the database, this is called the deletion anomaly in the database.

Anomalies in DBMS

Worker_id	Worker_name	Worker_dept	Worker_address
65	Ramesh	ECT001	Jaipur
65	Ramesh	ECT002	Jaipur
73	Amit	ECT002	Delhi
76	Vikas	ECT501	Pune
76	Vikas	ECT502	Pune
79	Rajesh	ECT669	Mumbai

Normalization Definition

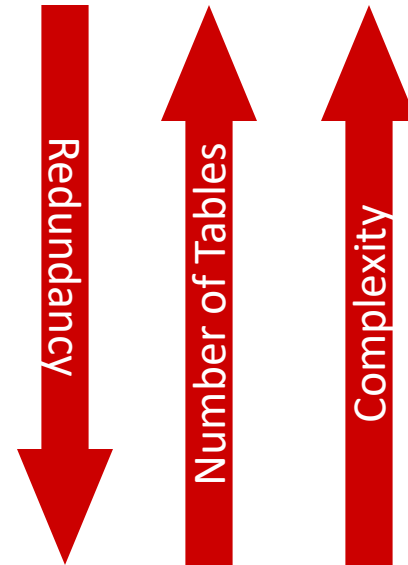
- This is the process which allows you to winnow out redundant data within your database.
- This involves restructuring the tables to successively meeting higher forms of Normalization.
- A properly normalized database should have the following characteristics
 - Scalar values in each fields
 - Absence of redundancy.
 - Minimal use of null values.
 - Minimal loss of information.

Normalizaytion Definition

- Redundant data wastes disk space and creates maintenance problems.
- If data that exists in more than one place must be changed, the data must be changed in exactly the same way in all locations.
- A customer address change is much easier to implement if that data is stored only in the Customers table and nowhere else in the database.

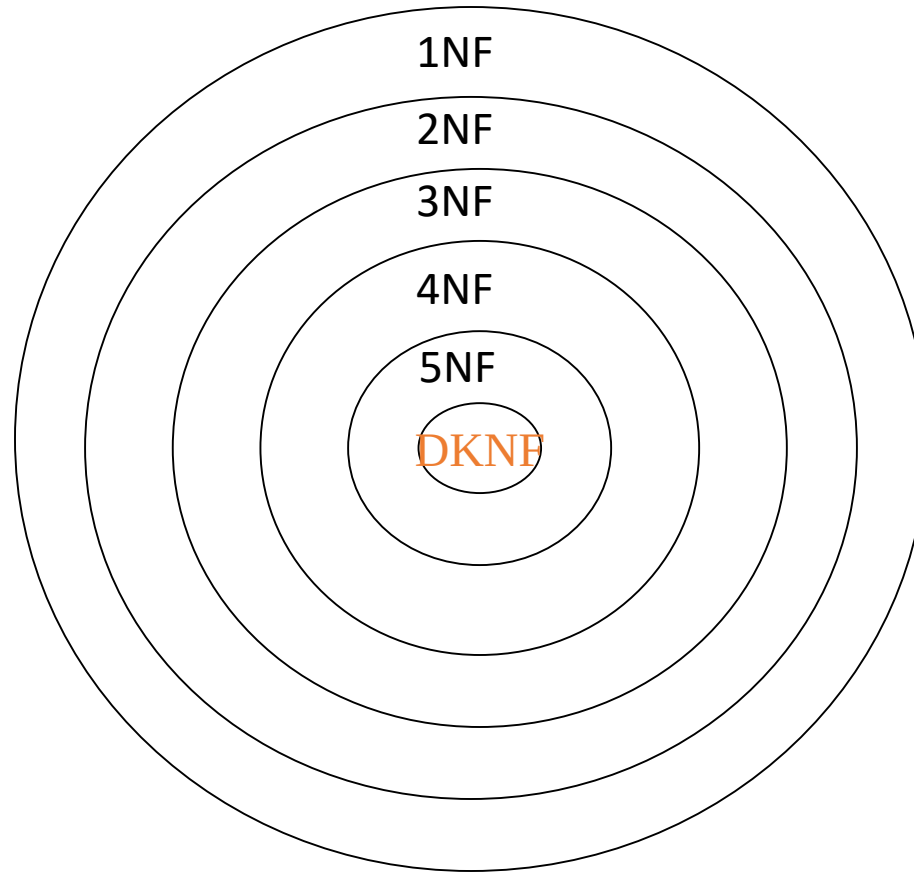
Levels of Normalization

- Levels of normalization based on the amount of redundancy in the database.
- Various levels of normalization are:
 - First Normal Form (1NF)
 - Second Normal Form (2NF)
 - Third Normal Form (3NF)
 - Boyce-Codd Normal Form (BCNF)
 - Fourth Normal Form (4NF)
 - Fifth Normal Form (5NF)
 - Domain Key Normal Form (DKNF)



Most databases should be 3NF or BCNF in order to avoid the database anomalies.

Levels of Normalization



Each higher level is a subset of the lower level

First Normal Form (1NF)

A table is considered to be in 1NF if all the fields contain only scalar values (as opposed to list of values).

Example (Not 1NF)

ISBN	Title	AuNam	AuPhone	PubName	PubPhone	Price
0-321-32132-1	Balloon	e Sleepy, Snoopy, Grumpy	321-321-1111, 232-234-1234, 665-235-6532	Small House	714-000-0000	\$34.00
0-55-123456-9	Main Street	Jones, Smith	123-333-3333, 654-223-3455	Small House	714-000-0000	\$22.95
0-123-45678-0	Ulysses	Joyce	666-666-6666	Alpha Press	999-999-9999	\$34.00
1-22-233700-0	Visual Basic	Roman	444-444-4444	Big House	123-456-7890	\$25.00

Author and AuPhone columns are not scalar

1NF - Decomposition

1. Place all items that appear in the repeating group in a new table
2. Designate a primary key for each new table produced.
3. Duplicate in the new table the primary key of the table from which the repeating group was extracted or vice versa.

Example (1NF)

ISBN	Title	PubName	PubPhone	Price
0-321-32132-1	Balloon	Small House	714-000-0000	\$34.00
0-55-123456-9	Main Street	Small House	714-000-0000	\$22.95
0-123-45678-0	Ulysses	Alpha Press	999-999-9999	\$34.00
1-22-233700-0	Visual Basic	Big House	123-456-7890	\$25.00

ISBN	AuName	AuPhone
0-321-32132-1	Sleepy	321-321-1111
0-321-32132-1	Snoopy	232-234-1234
0-321-32132-1	Grumpy	665-235-6532
0-55-123456-9	Jones	123-333-3333
0-55-123456-9	Smith	654-223-3455
0-123-45678-0	Joyce	666-666-6666
1-22-233700-0	Roman	444-444-4444

Functional Dependencies

1. If one set of attributes in a table determines another set of attributes in the table, then the second set of attributes is said to be functionally dependent on the first set of attributes.

Example 1

ISBN	Title	Price
0-321-32132-1	Balloon	\$34.00
0-55-123456-9	Main Street	\$22.95
0-123-45678-0	Ulysses	\$34.00
1-22-233700-0	Visual Basic	\$25.00

Table Scheme: {ISBN, Title, Price}

Functional Dependencies: {ISBN} $\rightarrow$ {Title}

{ISBN} $\rightarrow$ {Price}

Functional Dependencies

Example 2

PubID	PubName	PubPhone
1	Big House	999-999-9999
2	Small House	123-456-7890
3	Alpha Press	111-111-1111

Table Scheme: {PubID, PubName, PubPhone}

Functional Dependencies: {PubId} $\rightarrow$ {PubPhone}

$$\{\text{PubId}\} \rightarrow \{\text{PubName}\}$$
$$\{\text{PubName}, \text{PubPhone}\} \rightarrow \{\text{PubID}\}$$

Example 3

AuID	AuName	AuPhone
1	Sleepy	321-321-1111
2	Snoopy	232-234-1234
3	Grumpy	665-235-6532
4	Jones	123-333-3333
5	Smith	654-223-3455
6	Joyce	666-666-6666
7	Roman	444-444-4444

Table Scheme: {AuID, AuName, AuPhone}

Functional Dependencies: $\{\text{Auld}\} \rightarrow \{\text{AuPhone}\}$

$$\{\text{Auld}\} \rightarrow \{\text{AuName}\}$$
$$\{\text{AuName}, \text{AuPhone}\} \rightarrow \{\text{AuID}\}$$

FD – Example

Database to track reviews of papers submitted to an academic conference. Prospective authors submit papers for review and possible acceptance in the published conference proceedings. Details of the entities

- Author information includes a unique author number, a name, a mailing address, and a unique (optional) email address.
- Paper information includes the primary author, the paper number, the title, the abstract, and review status (pending, accepted, rejected)
- Reviewer information includes the reviewer number, the name, the mailing address, and a unique (optional) email address
- A completed review includes the reviewer number, the date, the paper number, comments to the authors, comments to the program chairperson, and ratings (overall, originality, correctness, style, clarity)

FD – Example

Functional Dependencies

- $\text{AuthNo} \rightarrow \text{AuthName}, \text{AuthEmail}, \text{AuthAddress}$
- $\text{AuthEmail} \rightarrow \text{AuthNo}$
- $\text{PaperNo} \rightarrow \text{Primary-AuthNo}, \text{Title}, \text{Abstract}, \text{Status}$
- $\text{RevNo} \rightarrow \text{RevName}, \text{RevEmail}, \text{RevAddress}$
- $\text{RevEmail} \rightarrow \text{RevNo}$
- $\text{RevNo}, \text{PaperNo} \rightarrow \text{AuthComm}, \text{Prog-Comm}, \text{Date}, \text{Rating1}, \text{Rating2}, \text{Rating3}, \text{Rating4}, \text{Rating5}$

Second Normal Form (2NF)

For a table to be in 2NF, there are two requirements

- The database is in first normal form
- All **nonkey** attributes in the table must be functionally dependent on the entire primary key

Or

- No partial dependency between non-key attributes

***Note:** Remember that we are dealing with non-key attributes*

Example 1 (Not 2NF)

Scheme $\rightarrow$ {Title, PubId, AuId, Price, AuAddress}

1. Key $\rightarrow$ {Title, PubId, AuId}
2. {Title, PubId, AuId} $\rightarrow$ {Price}
3. {AuId} $\rightarrow$ {AuAddress}
4. AuAddress does not belong to a key
5. AuAddress functionally depends on AuId which is a subset of a key

Second Normal Form (2NF)

Example 2 (Not 2NF)

Scheme $\rightarrow$ {City, Street, HouseNumber, HouseColor, CityPopulation}

1. key $\rightarrow$ {City, Street, HouseNumber}
2. {City, Street, HouseNumber} $\rightarrow$ {HouseColor}
3. {City} $\rightarrow$ {CityPopulation}
4. CityPopulation does not belong to any key.
5. CityPopulation is functionally dependent on the City which is a proper subset of the key

Example 3 (Not 2NF)

Scheme $\rightarrow$ {studio, movie, budget, studio_city}

1. Key $\rightarrow$ {studio, movie}
2. {studio, movie} $\rightarrow$ {budget}
3. {studio} $\rightarrow$ {studio_city}
4. studio_city is not a part of a key
5. studio_city functionally depends on studio which is a proper subset of the key

2NF - Decomposition

1. If a data item is fully functionally dependent on only a part of the primary key, move that data item and that part of the primary key to a new table.
2. If other data items are functionally dependent on the same part of the key, place them in the new table also
3. Make the partial primary key copied from the original table the primary key for the new table. Place all items that appear in the repeating group in a new table

Example 1 (Convert to 2NF)

Old Scheme → {Title, PubId, AuId, Price, AuAddress}

New Scheme → {Title, PubId, AuId, Price}

New Scheme → {AuId, AuAddress}

2NF - Decomposition

Example 2 (Convert to 2NF)

Old Scheme → {Studio, Movie, Budget, StudioCity}

New Scheme → {Movie, Studio, Budget}

New Scheme → {Studio, City}

Example 3 (Convert to 2NF)

Old Scheme → {City, Street, HouseNumber, HouseColor, CityPopulation}

New Scheme → {City, Street, HouseNumber, HouseColor}

New Scheme → {City, CityPopulation}

Example

Teacher_Id	Subject	Teacher_Age
111	Maths	38
111	Physics	38
222	Biology	38
333	Physics	40
333	Chemistry	40

Teacher_Id	Teacher_Age
111	38
222	38
333	40

Teacher_Id	Subject
111	Maths
111	Physics
222	Biology
333	Physics
333	Chemistry

Third Normal Form (3NF)

This form dictates that all **non-key** attributes of a table must be functionally dependent on a candidate key i.e. there can be no interdependencies among non-key attributes.

For a table to be in 3NF, there are two requirements

- The table should be second normal form
- No attribute is transitively dependent on the primary key

Or

- All non-key attribute must be directly functionally dependent on the PK

Example (Not in 3NF)

Scheme $\rightarrow$ {Title, PubID, PageCount, Price }

1. Key $\rightarrow$ {Title, PubID}
2. {Title, PubID} $\rightarrow$ {PageCount}
3. {PageCount} $\rightarrow$ {Price}
4. Both Price and PageCount depend on a key hence 2NF
5. Transitively {Title, PubID} $\rightarrow$ {Price} hence not in 3NF

Third Normal Form (3NF)

Example 2 (Not in 3NF)

Scheme $\rightarrow$ {Studio, StudioCity, CityTemp}

1. Primary Key $\rightarrow$ {Studio}
2. {Studio} $\rightarrow$ {StudioCity}
3. {StudioCity} $\rightarrow$ {CityTemp}
4. {Studio} $\rightarrow$ {CityTemp}
5. Both StudioCity and CityTemp depend on the entire key hence 2NF
6. CityTemp transitively depends on Studio hence violates 3NF

Example 3 (Not in 3NF)

Scheme $\rightarrow$ {BuildingID, Contractor, Fee}

1. Primary Key $\rightarrow$ {BuildingID}
2. {BuildingID} $\rightarrow$ {Contractor}
3. {Contractor} $\rightarrow$ {Fee}
4. {BuildingID} $\rightarrow$ {Fee}
5. Fee transitively depends on the BuildingID
6. Both Contractor and Fee depend on the entire key hence 2NF

BuildingID	Contractor	Fee
100	Randolph	1200
150	Ingersoll	1100
200	Randolph	1200
250	Pitkin	1100
300	Randolph	1200

3NF - Decomposition

1. Move all items involved in transitive dependencies to a new entity.
2. Identify a primary key for the new entity.
3. Place the primary key for the new entity as a foreign key on the original entity.

Example 1 (Convert to 3NF)

Old Scheme → {Title, PubID, PageCount, Price }

New Scheme → {PubID, Title, Price}

New Scheme → {Title, PageCount}

3NF - Decomposition

Example 2 (Convert to 3NF)

Old Scheme → {Studio, StudioCity, CityTemp}

New Scheme → {Studio, StudioCity}

New Scheme → {StudioCity, CityTemp}

Example 3 (Convert to 3NF)

Old Scheme → {BuildingID, Contractor, Fee}

New Scheme → {BuildingID, Contractor}

New Scheme → {Contractor, Fee}

BuildingID	Contractor
100	Randolph
150	Ingersoll
200	Randolph
250	Pitkin
300	Randolph

Contractor	Fee
Randolph	1200
Ingersoll	1100
Pitkin	1100

Emp_Id	Emp_Name	Emp_Zip	Emp_State	Emp_City	Emp_District
1001	John	282005	UP	Agra	Dayal Bagh
1002	Ajeet	222008	TN	Chennai	M-City
1006	Lora	282007	TN	Chennai	Urrapak kam
1101	Lilly	292008	UK	Pauri	Bhagwan
1201	Steve	222999	MP	Gwalior	Ratan

Emp_Id	Emp_Name	Emp_Zip
1001	John	282005
1002	Ajeet	222008
1006	Lora	282007
1101	Lilly	292008
1201	Steve	222999

Emp_Zip	Emp_State	Emp_City	Emp_District
282005	UP	Agra	Dayal Bagh
222008	TN	Chennai	M-City
282007	TN	Chennai	Urrapakkam
292008	UK	Pauri	Bhagwan
222999	MP	Gwalior	Ratan

Super keys: {Emp_Id}, {Emp_Id, Emp_Name}, {Emp_Id, Emp_Name, Emp_Zip}...SO ON

Candidate Keys: {Emp_Id}

Non-prime attributes: all attributes except Emp_Id are non-prime as they are not part of any candidate keys.

Boyce-Codd Normal Form (BCNF)

- BCNF does not allow dependencies between attributes that belong to candidate keys.
- BCNF is a refinement of the third normal form in which it drops the restriction of a non-key attribute from the 3rd normal form.
- Third normal form and BCNF are not same if the following conditions are true:
 - The table has two or more candidate keys
 - Each attribute can be defined by its candidate key
 - At least two of the candidate keys are composed of more than one attribute
 - The keys are not disjoint i.e. The composite candidate keys share some attributes

Example 1 - Address (Not in BCNF)

Scheme $\rightarrow$ {City, Street, ZipCode }

1. Key1 $\rightarrow$ {City, Street }
2. Key2 $\rightarrow$ {ZipCode, Street}
3. No non-key attribute hence 3NF
4. {City, Street} $\rightarrow$ {ZipCode}
5. {ZipCode} $\rightarrow$ {City}
6. Dependency between attributes belonging to a key

Boyce Codd Normal Form (BCNF)

Example 2 - Movie (Not in BCNF)

Scheme $\rightarrow$ {MovieTitle, MovieID, PersonName, Role, Payment }

1. Key1 $\rightarrow$ {MovieTitle, PersonName}
2. Key2 $\rightarrow$ {MovieID, PersonName}
3. Both role and payment functionally depend on both candidate keys thus 3NF
4. {MovieID} $\rightarrow$ {MovieTitle}
5. Dependency between MovieID & MovieTitle Violates BCNF

Example 3 - Consulting (Not in BCNF)

Scheme $\rightarrow$ {Client, Problem, Consultant}

1. Key1 $\rightarrow$ {Client, Problem}
2. Key2 $\rightarrow$ {Client, Consultant}
3. No non-key attribute hence 3NF
4. {Client, Problem} $\rightarrow$ {Consultant}
5. {Client, Consultant} $\rightarrow$ {Problem}
6. Dependency between attributes belonging to keys violates BCNF

BCNF - Decomposition

1. Place the two candidate primary keys in separate entities
2. Place each of the remaining data items in one of the resulting entities according to its dependency on the primary key.

Example 1 (Convert to BCNF)

Old Scheme $\rightarrow$ {City, Street, ZipCode }

New Scheme1 $\rightarrow$ {ZipCode, Street}

New Scheme2 $\rightarrow$ {City, Street}

- Loss of relation {ZipCode} $\rightarrow$ {City}
- Alternate New Scheme1 $\rightarrow$ {ZipCode, Street }
- Alternate New Scheme2 $\rightarrow$ {ZipCode, City}

Decomposition – Loss of Information

1. If decomposition does not cause any loss of information it is called a **lossless** decomposition.
2. If a decomposition does not cause any dependencies to be lost it is called a **dependency-preserving** decomposition.
3. Any table scheme can be decomposed in a lossless way into a collection of smaller schemas that are in BCNF form. However the dependency preservation is not guaranteed.
4. Any table can be decomposed in a lossless way into 3rd normal form that also preserves the dependencies.
 - 3NF may be better than BCNF in some cases

BCNF - Decomposition

Example 2 (Convert to BCNF)

Old Scheme $\rightarrow$ {MovieTitle, MovieID, PersonName, Role, Payment }

New Scheme $\rightarrow$ {MovieID, PersonName, Role, Payment}

New Scheme $\rightarrow$ {MovieTitle, PersonName}

- Loss of relation {MovieID} $\rightarrow$ {MovieTitle}

New Scheme $\rightarrow$ {MovieID, PersonName, Role, Payment}

New Scheme $\rightarrow$ {MovieID, MovieTitle}

- We got the {MovieID} $\rightarrow$ {MovieTitle} relationship back

Example 3 (Convert to BCNF)

Old Scheme $\rightarrow$ {Client, Problem, Consultant}

New Scheme $\rightarrow$ {Client, Consultant}

New Scheme $\rightarrow$ {Client, Problem}

Functional dependencies in the table above:

Emp_Id -> Emp_Nationality

Emp_Dept -> {Dept_Type, Dept_No_Of_Emp}

Candidate key: {Emp_Id, Emp_Dept}

Emp_Id	Emp_Nationality	Emp_Dept	Dept_Type	Dept_No_Of_Emp
1001	Austrian	Production and planning	D001	200
1001	Austrian	stores	D001	250
1002	American	design and technical support	D134	100
1002	American	Purchasing department	D134	600

Emp_Id	Emp_Nationality
1001	Austrian
1002	American

Emp_Dept	Dept_Type	Dept_No_Of_Emp
Production and planning	D001	200
stores	D001	250
design and technical support	D134	100
Purchasing department	D134	600

Emp_Id	Emp_Dept
1001	Production and planning
1001	stores
1002	design and technical support
1002	Purchasing department

Fourth Normal Form (4NF)

- Fourth normal form eliminates independent many-to-one relationships between columns.
- To be in Fourth Normal Form,
 - a relation must first be in Boyce-Codd Normal Form.
 - a given relation may not contain more than one multi-valued attribute.

Example (Not in 4NF)

Scheme $\rightarrow$ {MovieName, ScreeningCity, Genre}

Primary Key: {MovieName, ScreeningCity, Genre}

1. All columns are a part of the only candidate key, hence BCNF
2. Many Movies can have the same Genre
3. Many Cities can have the same movie
4. Violates 4NF

Movie	ScreeningCity	Genre
Hard Code	Los Angeles	Comedy
Hard Code	New York	Comedy
Bill Durham	Santa Cruz	Drama
Bill Durham	Durham	Drama
The Code Warrior	New York	Horror

Fourth Normal Form (4NF)

Example 2 (Not in 4NF)

Scheme → {Manager, Child, Employee}

1. Primary Key → {Manager, Child, Employee}
2. Each manager can have more than one child
3. Each manager can supervise more than one employee
4. 4NF Violated

Manager	Child	Employee
Jim	Beth	Alice
Mary	Bob	Jane
Mary	NULL	Adam

Example 3 (Not in 4NF)

Scheme → {Employee, Skill, ForeignLanguage}

1. Primary Key → {Employee, Skill, Language }
2. Each employee can speak multiple languages
3. Each employee can have multiple skills
4. Thus violates 4NF

Employee	Skill	Language
1234	Cooking	French
1234	Cooking	German
1453	Carpentry	Spanish
1453	Cooking	Spanish
2345	Cooking	Spanish

4NF - Decomposition

1. Move the two multi-valued relations to separate tables
2. Identify a primary key for each of the new entity.

Example 1 (Convert to 3NF)

Old Scheme → {MovieName, ScreeningCity, Genre}

New Scheme → {MovieName, ScreeningCity}

New Scheme → {MovieName, Genre}

Movie	Genre
Hard Code	Comedy
Bill Durham	Drama
The Code Warrior	Horror

Movie	ScreeningCity
Hard Code	Los Angeles
Hard Code	New York
Bill Durham	Santa Cruz
Bill Durham	Durham
The Code Warrior	New York

4NF - Decomposition

Example 2 (Convert to 4NF)

Old Scheme → {Manager, Child, Employee}

New Scheme → {Manager, Child}

New Scheme → {Manager, Employee}

Manager	Child
Jim	Beth
Mary	Bob

Manager	Employee
Jim	Alice
Mary	Jane
Mary	Adam

Example 3 (Convert to 4NF)

Old Scheme → {Employee, Skill, ForeignLanguage}

New Scheme → {Employee, Skill}

New Scheme → {Employee, ForeignLanguage}

Employee	Skill
1234	Cooking
1453	Carpentry
1453	Cooking
2345	Cooking

Employee	Language
1234	French
1234	German
1453	Spanish
2345	Spanish

Fifth Normal Form (5NF)

- Fifth normal form is satisfied when all tables are broken into as many tables as possible in order to avoid redundancy. Once it is in fifth normal form it cannot be broken into smaller relations without changing the facts or the meaning.

Domain Key Normal Form (DKNF)

- The relation is in DKNF when there can be no insertion or deletion anomalies in the database.